

25 DSA Patterns — Master Sheet

Top 5 practice problems + one-line core logic for each pattern | Product-based company interview prep

How to use this: for each pattern, read the one-line logic first and try to state it in your own words before you open a problem. Solve the 5 problems in order (they go easy → hard). If you can solve #4 and #5 without hints, the pattern is locked in — move to the next one.

Arrays & Strings

1. Two Pointers

Logic: Move two indices toward each other (or in the same direction) based on how the current pair compares to the target, shrinking the search space every step.

- Two Sum II – Input Array Is Sorted
- 3Sum
- Container With Most Water
- Trapping Rain Water
- Remove Duplicates from Sorted Array

2. Sliding Window

Logic: Grow the window from the right; shrink it from the left the moment it breaks the constraint, tracking the best window seen.

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Longest Repeating Character Replacement
- Max Consecutive Ones III
- Sliding Window Maximum

3. Prefix Sum / Difference Array

Logic: Precompute running sums so any range sum becomes one subtraction, turning repeated $O(n)$ range queries into $O(1)$.

- Subarray Sum Equals K
- Range Sum Query – Immutable
- Product of Array Except Self
- Continuous Subarray Sum
- Contiguous Array

4. Kadane's Algorithm

Logic: At each position, decide whether to extend the running subarray or restart from here, keeping the best sum seen so far.

- Maximum Subarray
- Maximum Product Subarray
- Maximum Sum Circular Subarray
- Best Time to Buy and Sell Stock
- Maximum Absolute Sum of Any Subarray

5. Cyclic Sort

Logic: Since values live in the range $1..n$, repeatedly swap each number to its “home” index until every slot holds the right value.

- Missing Number
- Find the Duplicate Number
- Find All Duplicates in an Array
- First Missing Positive
- Set Mismatch

Linked List

6. Fast & Slow Pointers

Logic: Move one pointer one step and another two steps; if they collide a cycle exists, and the meeting point locates it.

- Linked List Cycle
- Linked List Cycle II
- Happy Number
- Middle of the Linked List
- Palindrome Linked List

7. In-place Reversal of Linked List

Logic: Walk the list once, flipping each node's next pointer backward using prev / curr / next temporary references.

- Reverse Linked List
- Reverse Linked List II
- Reverse Nodes in k-Group
- Swap Nodes in Pairs
- Rotate List

Stack

8. Monotonic Stack

Logic: Keep a stack in strictly increasing or decreasing order, popping whenever the new element breaks that order to resolve "next greater/smaller."

- Next Greater Element I
- Next Greater Element II
- Daily Temperatures
- Largest Rectangle in Histogram
- Remove K Digits

Binary Search

9. Binary Search (classic + on answer space)

Logic: Repeatedly halve the search space by testing the midpoint against a condition that flips exactly once across the range.

- Search in Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Koko Eating Bananas
- Capacity To Ship Packages Within D Days
- Median of Two Sorted Arrays

Heap / Priority Queue

10. Top 'K' Elements

Logic: Maintain a heap capped at size K, pushing new elements and popping the weakest so it always holds the current top K.

- Kth Largest Element in an Array
- Top K Frequent Elements
- K Closest Points to Origin
- Task Scheduler
- Reorganize String

11. Two Heaps

Logic: Balance a max-heap of the smaller half against a min-heap of the larger half so the median always sits on top of one of them.

- Find Median from Data Stream
- Sliding Window Median
- IPO
- Kth Largest Element in a Stream
- Maximum Frequency Stack

12. K-way Merge

Logic: Seed a min-heap with the first element of every list, then repeatedly pop the smallest and push its list's next element.

- Merge k Sorted Lists
- Kth Smallest Element in a Sorted Matrix
- Smallest Range Covering Elements from K Lists
- Find K Pairs with Smallest Sums
- Merge Sorted Array

Trees

13. Tree BFS (Level Order)

Logic: Process nodes level by level with a queue, using the queue's size at each step to know where one level ends and the next begins.

- Binary Tree Level Order Traversal
- Binary Tree Zigzag Level Order Traversal
- Binary Tree Right Side View
- Minimum Depth of Binary Tree
- Populating Next Right Pointers in Each Node

14. Tree DFS (Recursive)

Logic: Recurse into children carrying state (sum, depth, path) downward, then combine the children's results back up at each node.

- Path Sum II
- Diameter of Binary Tree
- Validate Binary Search Tree
- Lowest Common Ancestor of a Binary Tree
- Binary Tree Maximum Path Sum

15. Trie (Prefix Tree)

Logic: Each node stores a map of character → child node plus an end-of-word flag, so shared prefixes share the same path from root.

- Implement Trie (Prefix Tree)
- Word Search II
- Design Add and Search Words Data Structure
- Replace Words
- Longest Word in Dictionary

Graphs

16. Graph BFS / DFS

Logic: Visit every reachable node exactly once via a queue or recursion, marking visited nodes so you never loop back.

- Number of Islands
- Clone Graph
- Rotting Oranges
- Pacific Atlantic Water Flow
- Word Ladder

17. Topological Sort

Logic: Repeatedly peel off nodes with zero incoming edges, appending them to the order and decrementing their neighbors' in-degree.

- Course Schedule
- Course Schedule II
- Alien Dictionary
- Sequence Reconstruction
- Minimum Height Trees

18. Union-Find (DSU)

Logic: Each node points toward a parent; union merges two roots, and path compression flattens future lookups toward $O(1)$.

- Number of Provinces
- Redundant Connection
- Accounts Merge
- Number of Islands II
- Graph Valid Tree

19. Dijkstra's Algorithm

Logic: Always expand the closest unvisited node from a min-heap next, relaxing neighbor distances whenever a shorter path is found.

- Network Delay Time
- Cheapest Flights Within K Stops
- Path With Minimum Effort
- Swim in Rising Water
- Path with Maximum Probability

Recursion & Combinatorics

20. Subsets / Permutations / Combinations

Logic: At each step choose to include or skip the next element (or pick the next unused one), recursing and backtracking through every branch.

- Subsets
- Subsets II
- Permutations
- Combinations
- Combination Sum

21. Backtracking

Logic: Build the solution one choice at a time, and the instant a partial path violates a constraint, undo it and try the next option.

- N-Queens
- Sudoku Solver
- Word Search
- Palindrome Partitioning
- Generate Parentheses

Intervals & Greedy

22. Merge Intervals

Logic: Sort intervals by start time, then merge whenever the next interval's start falls before or at the current merged interval's end.

- Merge Intervals
- Insert Interval
- Non-overlapping Intervals
- Meeting Rooms II
- Interval List Intersections

23. Greedy

Logic: Make the locally best choice at each step without revisiting past decisions, relying on the problem's structure to guarantee a global optimum.

- Jump Game
- Jump Game II
- Gas Station
- Partition Labels
- Candy

Bit Manipulation

24. Bit Manipulation

Logic: Use XOR to cancel matching pairs and AND / OR / shift to isolate or set individual bits, since every number is just a bit pattern underneath.

- Single Number
- Single Number II
- Number of 1 Bits
- Counting Bits
- Sum of Two Integers

Dynamic Programming

25. Dynamic Programming (1D / Knapsack / Grid / LCS-LIS)

Logic: Define a state that captures the subproblem, find the recurrence linking it to smaller states, then build up from base cases with memoization or a table.

- Climbing Stairs
- House Robber
- Coin Change
- Longest Common Subsequence
- Partition Equal Subset Sum

Suggested Study Order

Phase 1 – Foundation	Two Pointers, Sliding Window, Fast & Slow Pointers, Binary Search, Prefix Sum, Cyclic Sort
Phase 2 – Stack & Heap	Monotonic Stack, Top K Elements, Two Heaps, K-way Merge, In-place LL Reversal
Phase 3 – Trees & Recursion	Tree BFS, Tree DFS, Trie, Subsets/Permutations, Backtracking
Phase 4 – Graphs	Graph BFS/DFS, Topological Sort, Union-Find, Dijkstra's Algorithm
Phase 5 – Intervals, Bits & DP	Merge Intervals, Greedy, Bit Manipulation, Dynamic Programming (all sub-types)